CHARITÉ
UNIVERSITÄTSMEDIZIN BERLIN

Erasmus Mundus Joint Master's Degree on Physiology and Medicine of Humans in Space and Extreme Environments – spacemed –

Containerizing a Scientific Data-Analysis Pipeline for HPC Deployment: Build, Data Binding, Parallelization, and GPU Acceleration with Apptainer

A technical report of the 'spacemed'-Pipeline on the Charité High-Performance Computing Cluster

UC6-EC4 – Independent Research Project Report

Kira Gruhlke
Matriculation Number
9252110

30.08.2026

Charité Universitätsmedizin Berlin, Charité GB-IT, Scientific Computing / HPC, Magnus Hagdorn

Table of Content

Abstract 3

1. Introduction 3
    1.1 Background 3
    1.2 Rationale 4

2. Methods 4
    2.1. Structure 4
    2.2 Hard- and Software 5
    2.3 Container Construction 5
    2.4 Data-Binding Strategy 8
    2.5 Parallelization Strategies 9
    2.6 GPU-Acceleration Setup 10

3. Results 11
    3.1 Container Build and Local Validation 11
    3.2 Data Binding on Workstation and Cluster 12
    3.3 Parallel Execution 12
    3.4 GPU-Accelerated Execution 13

4. Discussion 13
    4.1 Portability and Reproducibility 13
    4.2 Code/Data Separation as an HPC Design Pattern 13
    4.3 Parallelization Trade-offs 14
    4.4 GPU Virtualization and the Reproducibility Compromise 14
    4.5 Limitations 15

5. Outlook and Perspectives 15

References 16

Academic integrity declaration 17
    Information on the use of AI-based tools 17

Appendix 18

Abstract

Reproducibility and portability are longstanding challenges in scientific computing, particularly when an analysis pipeline developed on a personal workstation must later run unmodified on a shared high-performance computing (HPC) cluster. This technical report documents the design and evaluation of a containerized deployment workflow for 'spacemed', a Python-based pipeline for analyzing physiological signals, built with Apptainer (formerly Singularity) and exercised end to end, from a local Debian 13 workstation to the Slurm-managed Charité Scientific Computing (SC) cluster.
The workflow built up step by step. A rootless, reproducible container was built from a minimal Debian base image using a Conda/Mamba environment, defined in a versioned .def file and executable without administrative privileges on the cluster. Code and data were then separated via bind mounts, allowing a single container to process arbitrary external datasets without being rebuilt, and keeping potentially sensitive measurements outside the shareable container artifact. Two complementary parallelization strategies for embarrassingly parallel workloads were evaluated next, host-level mpirun invocations on the workstation and Slurm array jobs on the cluster, both requiring no modification to the container itself. Finally, GPU-accelerated execution was enabled through Apptainer's --nv and --rocm mechanisms and validated with a minimal PyTorch test container across the cluster's NVIDIA and AMD GPU partitions.
The results indicate that host-level parallelization and runtime driver bind-mounting are practical, low-maintenance strategies for containerized scientific software on shared HPC infrastructure. Apptainer's central premise, that a single, immutable SIF image can be moved to the cluster and executed there without modification, held throughout all steps. The report also surfaces open questions around systematic performance benchmarking and data-governance procedures for sensitive datasets, which remain to be addressed in future work.

1. Introduction

1.1 Background

Container technologies have become a standard way of packaging scientific software together with its runtime dependencies so that an analysis can be executed identically on different machines. Apptainer (formerly Singularity) was designed specifically to address this: It packages a whole operating system and software environment (decided upon during the build) into a single, read-only Singularity Image Format (SIF) file that can be copied between systems like any other file (Kurtzer, 2017). Because a SIF image is immutable once built, it functions as a frozen, versionable artifact: the same file that was verified on a workstation can be transferred to a cluster and executed there without modification, which is guaranteeing for it to run identically for everyone who calls it (Kurtzer, 2017).
For the reasoning above, a container build with Apptainer is isolated from the user’s system. However, there are exceptions needed for proper functioning. These should still not break the main reason for Apptainer usage: compatibility.
A distinguishing design choice of Apptainer, relevant throughout this report, is that the host's home directory, temporary directories, and several system paths are bound into the container automatically, while any other host location, such as a project-directories on an High-Performance Computing (HPC) file system, must be bound explicitly (Apptainer Project, n.d.). This default behaviour shapes how code and data can be organized without sacrificing portability (David Moreau, 2023).

1.2 Rationale

The pipeline used as a case report, ‘spacemed’, is a Python package for scientific analysis of physiological signals. The package is installable via pip and depends on a Conda environment defined in a ‘spacemed.yml’ file shipped with the repository (Gruhlke, 2026).
Local development of the Apptainers took place on a personal Debian 13 workstation while production runs were intended and later on deployed on the Charité Scientific Computing (SC) cluster, a Slurm-managed HPC (High Performance Computing) system reachable through three interchangeable frontend nodes and a heterogeneous set of CPU and GPU compute nodes (Charité Universitätsmedizin Berlin, n.d.). Here the Apptainers were deployed and tested.
Moving such a pipeline from a workstation to a shared cluster rose four practical questions that motivated this work.
First, the analysis environment must be captured once and reproduced exactly on the cluster, without requiring root access there (Section 2.3).
Second, because the underlying data may be sensitive and is expected to change from run to run, it cannot be baked into the container image but must be attached at run time from an external location (Section 2.4).
Third, many of the analyses of interest are many analyses of the same kind that can run completely independently (Section 2.5).
Fourth, forthcoming workloads are expected to require GPU acceleration, which conflicts in principle with a container's isolation from the host, since GPU drivers are tied to the host kernel and physical hardware and cannot simply be packaged inside a frozen image (Section 2.6).
This technical report gives an overview of a few possible ways on how to address these questions. Together with the project published on the Charité-Cluster, GitLab or GitHub (Gruhlke, 2026) it is a user’s guide on how to use Apptainer on an HPC-Cluster on the example of the ‘spacemed’-project (Python-based) on the Charité’s Scientific Computing cluster.

2. Methods

2.1. Structure

The work was organized as four sequential, cumulative work packages, each building on the previous step:

1. constructing a portable container for the ‘spacemed’- environment (spacemed26.sif; Section 2.3)
2. separating code from data with the help of bind mounts (spacemed26_data.sif; Section 2.4)
3. host-level parallel execution (spacemed26_data.sif; Section 2.5)
4. enabling GPU-accelerated execution using a minimal PyTorch test container as a stand-in for future GPU-dependent analyses (gu_test_nvidia.sif, gpu_test_amd.sif; Section 2.6).

Each work package followed the same general pattern: implement the change on the local workstation first, verify it there, then transfer and re-verify it on the Charité SC cluster.

2.2 Hard- and Software

Local development took place on a Debian 13 (trixie) workstation running the KDE Plasma desktop, on an ASUS laptop (Q324UAK Notebook PC) without a dedicated NVIDIA or AMD GPU; a Surface tablet (Microsoft Surface Pro 11th edition) without a discrete GPU was used as a secondary test system.
The Charité SC cluster is reached through three interchangeable frontend nodes (s-sc-frontend1–3), which run Rocky Linux 9.5, provide SSH login and job submission, and are strictly limited to light interactive use (1 GB of RAM); actual computation takes place on a separate pool of compute and GPU nodes managed by the Slurm workload manager. Slurm (Simple Linux Utility for Resource Management) is the workload manager that allocates hardware resources (CPUs, GPUs, RAM, etc.) between jobs and users. It decides when which job runs on which nodes. All nodes share a single home-directory file system with a 100 GB soft quota. The GPU compute pool comprises multiple NVIDIA partitions and a single-node AMD partition (see Appendix Table 1) (Charité Universitätsmedizin Berlin, n.d.; Charité Universitätsmedizin Berlin, n.d.).

Apptainer was installed in its rootless, non-setuid configuration, which additionally requires uidmap (to provide user namespaces via newuidmap/newgidmap), squashfs-tools (to build and read the compressed, read-only SquashFS layer that a SIF image embeds), and fuse-overlayfs (to provide a writable overlay for the container's read-only file system without a setuid helper) (Apptainer Project, n.d.; Naweiluo Zhou, 2022).

The base operating system for the container image in all containers was debian:trixie, taken from Docker Hub. The scientific Python environment was installed with Miniforge/Mamba, using the ‘spacemed.yml’ environment specification (Python 3.12, pandas, NumPy, SciPy, Matplotlib, Seaborn, Sphinx, Jupyter/Jupytext, and nibabel via pip) together with the ‘spacemed’ package itself, installed from the repository. Four containers were built in total: two including the ‘spacemed’-project, one of them without the example data to demonstrate the bind mount and two other PyTorch-based ones independent of the ‘spacemed’-project built once against CUDA for the NVIDIA partitions and once against ROCm for the AMD partition for the GPU experiments.

2.3 Container Construction

Containers are files that package an entire application together with everything it needs to run, so that the program can be executed on any compatible machine without being installed directly into the host operating system. This makes them relatively compact (compared to a full virtual environment) and easy to share (Kurtzer, 2017). They are compatible with any system running the hardware architecture, e.g. CPU x86_64. A container built for x86_64 will not run on an ARM machine, and a Linux container does not run natively on Windows or macOS without a virtualization layer underneath. This compatibility comes from the fact that the container uses the host's kernel directly; it does not bring its own kernel (Kurtzer, 2017).

Apptainer is a standard for scientific computing, particularly on HPC clusters (Kurtzer, 2017; Naweiluo Zhou, 2022). It allows the userland of an entire operating system, the filesystem, libraries, package manager, and system tools of a given distribution (e.g., Debian:trixie), to be packaged into a single .sif file, which significantly improves portability across different systems (Apptainer Project, n.d.). The kernel itself still comes from the host at runtime. This shared-kernel architecture is exactly what makes containers lightweight compared to a full

virtual machine, which runs an entirely separate kernel of its own (Naweiluo Zhou, 2022). In addition, Apptainer can run under normal user permissions instead of requiring root access, which makes it well suited for secure, shared use on multi-user public compute clusters (Naweiluo Zhou, 2022).

What belongs inside a container (David Moreau, 2023):

1. Operating system userland/distribution
2. Runtime environment (e.g., Python)
3. The actual program and its dependencies

Building a container requires root privileges, so the build process cannot be carried out on the cluster itself and must instead be done locally (Naweiluo Zhou, 2022). The build instructions are defined in a .def file, which specifies the components above (1-3) and where each should be obtained from - either locally or from a remote source. The .def file can also define runtime behavior, such as configuring the container to drop directly into the Python environment rather than requiring it to be invoked explicitly. Running the .def file produces the reproducible .sif file -the container itself (Apptainer Project, n.d.).

The container was specified with an Apptainer definition (.def) file, which pairs a header (Bootstrap: docker, From: debian:trixie) with a sequence of build-time sections (Apptainer Project, n.d.-a). The %post section, executed as root at build time, installed system packages required for the remaining steps (git, wget, ca-certificates), downloaded and installed Miniforge to /opt/conda in batch (non-interactive) mode, built the Conda environment from ‘spacemed.yml’ using the full path to the Mamba binary and installed the ‘spacemed’ package into that environment with the environment-specific pip binary. The %environment section then exposed the Conda environment's binaries on PATH for every subsequent invocation of the container, and a %runscript section used exec together with the full interpreter path to replace the container's entry process directly with Python, ensuring that interactive signals such as Ctrl+C are delivered correctly. For more detailed information see Figure 1.

```
Bootstrap: docker
From: debian:trixie

%post
    apt-get update && apt-get install -y git wget ca-certificates
    wget https://github.com/conda-
forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh -O
/tmp/miniforge.sh
    bash /tmp/miniforge.sh -b -p /opt/conda
    rm /tmp/miniforge.sh
    git clone https://github.com/KiraGruhlke/spacemed.git /opt/spacemed
    cd /opt/spacemed
    /opt/conda/bin/mamba env create -f spacemed.yml
    /opt/conda/envs/spacemed26/bin/pip install /opt/spacemed
    rm -rf /opt/spacemed/build

%environment
    export PATH=/opt/conda/envs/spacemed26/bin:$PATH

%runscript
    exec /opt/conda/envs/spacemed26/bin/python "$@"
```

Figure 1. Apptainer definition file (spacemed.def) used to build the 'spacemed' container.

In a later iteration (spacemed26_data.def), the initial git clone step in %post was replaced by a %files instruction that copies a local, pre-cleaned working copy of the repository (with the data directory already removed) into the image before %post runs, avoiding both a dependency on network access during the build and the possibility of the data directory being embedded in the image (Apptainer Project, n.d.).

The image was built locally with apptainer build (using --fakeroot where an administrator-configured setuid build was unavailable) and verified by opening an interactive shell inside the built image and confirming that the package could be imported and that its command-line entry points and job scripts, bundled under /opt/'spacemed'/jobs, were present and executable. The resulting .sif file was then copied to the cluster with rsync and re-verified there. For verification on the cluster, jobs that were already included in the container and ran interactively, were transformed into SLURM-Jobs (the Charité workload manager) to make them be able to be handed in through the frontend directly (see Figure 2).

```
#!/bin/bash
#SBATCH --job-name="plot heart rate"
#SBATCH --partition=compute
#SBATCH --nodes=1
#SBATCH --cpus-per-task=10
#SBATCH --mem=40G
#SBATCH --time=01:00:00
#SBATCH --output=/home/kigr11/Apptainer/spacemed/slurm-%j.out

pwd
apptainer exec /home/kigr11/Apptainer/spacemed/spacemed26.sif sm_plot_heart
/opt/spacemed/data/pulse_data.csv -o /home/kigr11/Apptainer/spacemed/results_phr.png
```

Figure 2. Slurm batch script (sbatch) for running the 'spacemed' container on the Charité SC HPC cluster.

For a more exhaustive overview of errors and diagnostics that occurred during this project, see Table 2 in the Appendix.

2.4 Data-Binding Strategy

Apptainer automatically binds a small, fixed set of host paths into every container unless the --contain/--containall flags are used (Apptainer Project, n.d.) (see Table 3):

Table 3. Automatically bound directories

Path	Purpose
$HOME (your Home-Directory)	To access your own files directly
/tmp	Temporary Files
/proc System Information (Processes)	technically necessary for the container to run
/sys	System information (hardware/kernel)
/dev	Device files (e.g., /dev/null, /dev/zero)
$PWD (current working directory)	To ensure relative paths work when called
/etc/hosts	Name resolution (which IP belongs to which hostname)
/etc/resolv.conf	DNS configuration

Any other host location, e.g. project directories typically used for larger data sets on an HPC file system, must be attached explicitly with the --bind flag within the sbatch-job or equivalent. A bind mount does not copy data. It is comparable to a link or a tunnel granting the container access to this specific directory for the time of its runtime. By default, it is read-and-write (:rw) to be able to store, for example results in the according directory but can also be set to read-only (:ro) for security reasons. Therefore, no additional disk space is consumed. (Apptainer Project, n.d.)

After the original container (spacemed26_data.sif) was built without embedded data, how external measurement data could be attached via a bind mount was tested, first locally, then on the cluster.
Locally, the first attempt bound the data directory directly to the path the code expects internally (/opt/spacemed/data). However, since plot_heart_rate.py takes the data path as a command-line argument rather than assuming a fixed location, a freely chosen mount point (/data) was used instead, with the file path passed explicitly at invocation (apptainer run --

bind ~/Dokumente/data:/data spacemed26_data.sif -m spacemed.plot_heart_rate /data/pulse_data.csv). This works because the .def file's %runscript passes all arguments ("$@") directly through to Python.

The same principle was then carried over to the cluster (see Figure 3). The output was deliberately saved under a new name (phr_bind.png) to distinguish it from the result of the first container, which had data built in, allowing both approaches to be compared.

```
#!/bin/bash
#SBATCH --job-name="plot heart rate bind"
#SBATCH --partition=compute
#SBATCH --nodes=1
#SBATCH --cpus-per-task=10
#SBATCH --mem=40G
#SBATCH --time=01:00:00
#SBATCH --output=/home/kigr11/Apptainer/spacemed_data/slurm-%j.out

pwd
apptainer exec --bind /home/kigr11/Data/apptainer_spacemed_data:/data
/home/kigr11/Apptainer/spacemed_data/spacemed26_data.sif sm_plot_heart
/data/pulse_data.csv -o /home/kigr11/Apptainer/spacemed_data/results_phr_bind.png
```

Figure 3. Revised Slurm batch script for the sm_plot_heart entry point, using an explicit --bind mount to attach external data at runtime.

2.5 Parallelization Strategies

Computer systems usually run one job after another. If I have multiple jobs of the same kind, that don't depend on each other (e.g. the same computational task on many separate slices of a single fMRI volume) I might want to speed the process by not running them one after another, but simultaneously. This is exactly what CPU-parallelization does: a batch of independent items can be processed without any communication between the individual computations.

Two strategies for task parallelization using multiple CPU cores exist:

- In-container MPI (Message Passing Interface): For this method MPI has to be installed directly within the container to allow it to run the same command on different data simultaneously. Parallelization takes place entirely inside the container (Apptainer Project, n.d.). This approach is more flexible and portable but potentially slower on a real cluster, as the MPI version within the container may not be able to utilize the cluster's hardware-specific optimizations (Krishna Kant Singh, 2026).

- Host-level parallelization: The container itself contains only the actual program (single-process) and remains without any awareness of parallelization. Distribution across multiple processes is handled by Slurm or MPI on the host system, outside the container: the container is simply launched multiple times in parallel, with ideally one instance per process or core (Apptainer Project, n.d.).
  This is the standard, recommended approach for HPC clusters, for two reasons:
  - The cluster often provides a highly optimized, hardware-specific version of MPI (e.g., for InfiniBand interconnects).
  - You do not need to maintain MPI libraries within the container that are precisely matched to the cluster hardware.

A practical advantage: no new container or modification of the `.def` file is required. Any existing container can simply be launched multiple times in parallel via 'mpirun' without needing to be adapted for MPI.

The local workstation tests trying to run the spacemed26_data.if multiple times were affected by two independent limits that had to be worked around explicitly:
OpenMPI's default refusal to start more processes than it detects available "slots"(free CPU cores) was worked around for testing purposes with --oversubscribe, or by specifying slots explicitly with --host localhost:N.
OpenMPI's attempt to pin every process to a distinct physical core, which fails once a laptop's small number of physical cores is exhausted, independently of its higher logical (hyperthreaded) core count reported by nproc was worked around with --bind-to none.

On the Slurm-managed cluster, both limits are resolved by the scheduler itself, which allocates a requested number of tasks and CPUs per task (#SBATCH --ntasks, --cpus-per-task) from genuinely free physical cores across the pool of compute nodes before the job starts (Charité Universitätsmedizin Berlin, n.d.). Building on this, the parallelization tested locally earlier was implemented as a full Slurm job (see Figure 4).

```
#!/bin/bash
#SBATCH --job-name="generate slices"
#SBATCH --partition=compute
#SBATCH --nodes=1
#SBATCH --cpus-per-task=10
#SBATCH --mem=40G
#SBATCH --time=01:00:00
#SBATCH --array=0-31
#SBATCH --output=/home/kigr11/Apptainer/spacemed_data/slurm-%A_%a.out

# the stuff we are going to do

pwd
apptainer exec --bind /home/kigr11/Data/apptainer_spacemed_data:/data
/home/kigr11/Apptainer/spacemed_data/spacemed26_data.sif sm_process_slice /data/3
_fMRI_TR2sec_3mm_3min.nii ${SLURM_ARRAY_TASK_ID}
/home/kigr11/Apptainer/spacemed_data/ps_output_${SLURM_ARRAY_TASK_ID}.png
```

Figure 4. Slurm array job for parallel processing of independent fMRI slices using the 'spacemed_data' container.

2.6 GPU-Acceleration Setup

CPUs are used when a task is complex and involves significant decision logic or dependencies between steps, or when several fully independent tasks need to be processed in parallel (e.g., 32 independent analysis runs across different datasets).

GPUs are worthwhile, when one task can be broken down into many similar, independent computational steps, that then need to be integrated into the task result, as in deep learning, image processing, or large-scale linear algebra.
HPC clusters offer GPU resources because many modern research applications rely on this particular kind of computational load: many similar, simple operations that can run independently and simultaneously. This is exactly what GPUs are built for.

The reason lies in the structural difference between CPUs and GPUs. A CPU has few but very powerful cores, well suited to complex, branching logic with many dependencies between steps. A GPU, by contrast, has thousands of simpler cores that are individually weaker but can execute the same simple operation in massive parallel.

A container is normally isolated from the host, but GPU acceleration is a deliberate exception to this isolation. The reason is that the NVIDIA or AMD driver is tightly bound to the host's kernel and to the physical GPU hardware, so it cannot simply be packaged inside a portable container image (Krishna Kant Singh, 2026). Apptainer handles this with the --nv flag for NVIDIA-GPUs (and the equivalent --rocm flag for AMD-GPUs): At runtime, Apptainer finds the necessary driver files on the host and makes them available inside the running container, without copying them into the image itself (comparable to the bind-mount command) (Apptainer Project, n.d.).

The CUDA- (or ROCm-) toolkit that the application actually uses stays fixed inside the container image. The only part that differs between systems is the driver, and drivers are kept backward compatible: a newer driver on the host can run an older CUDA version inside the container, but not the other way around (NVIDIA Corporation, n.d.). AMD's ROCm stack follows a similar but somewhat more symmetric model, maintaining both forward and backward compatibility between the GPU driver and the ROCm user-space software within a defined version window (AMD, n.d.).

Two variants of a minimal PyTorch test container (gpu_test_nvidia.sif, gpu_test_amd.sif) were built for validation, one linked against CUDA for the cluster's NVIDIA partitions (gpu, pgpu) and one linked against ROCm for the single-node AMD partition (agpu), since the two GPU software ecosystems are mutually incompatible and cannot coexist in one Python environment. The container was first built and executed locally, where consistent with the workstation's lack of a dedicated GPU PyTorch was successfully installed but reported no CUDA device available, confirming that the container itself was viable before it was tested with real hardware. On the cluster, GPU jobs were submitted as Slurm batch scripts requesting the appropriate partition and a GPU resource (--partition=gpu / agpu, --gres=gpu:1), and the container was executed with apptainer exec --nv (or --rocm for the AMD node).

3. Results

3.1 Container Build and Local Validation

The first 'spacemed' container was built on the workstation from the debian:trixie base image, using a .def file that cloned the repository directly from GitHub during the %post step.

Inside an interactive shell, import spacemed resolved correctly, and a peak-detection call against the bundled pulse-oximetry recording returned a plausible list of detected peaks, confirming the environment and package worked as intended.

Running scripts via apptainer run required calling the plotting script as a module (python -m spacemed.plot_heart_rate) rather than as a path. This avoided the necessity of the container to have access to a Graphical User Interface (GUI) in the plot-function and saved the output directly.

The validated container was then deployed to the cluster and tested via sbatch with phr.sh, producing both a job log (`slurm-%j.out`) and output image (`results_phr.png`) with no errors.

3.2 Data Binding on Workstation and Cluster

A second container (spacemed_data.sif) was built to remove the dataset from the image itself and instead access data through a bind mount to an external directory, avoiding the need to embed data inside the container. Rather than cloning from GitHub, the repository was first cloned locally, the data folder removed from this local copy, and the .def file rewritten to use %files instead of git clone in %post, copying the local source (/home/kira/Dokumente/spacemed) into the image at /opt/spacemed. This produced a smaller, data-independent image whose only dependency on the runtime environment is the external directory supplied at execution time via a bind mount, matching the intended deployment pattern for the HPC cluster.

For testing/running the same script mentioned in 3.1 was used both locally and on the cluster. The job phr.sh was supplemented by the bind mount command and now called phr_bind.sh. Both the job log (`slurm-%j.out`) and the resulting output (`results_phr_bind.png`) were saved automatically and produced successfully, with no errors reported locally as well as on the cluster.

3.3 Parallel Execution

Host-level parallelization was demonstrated first on the workstation using the spacemed_data.sif. The job script called the function ‘process_slice’, with each array task processing one slice of an fMRI NIfTI volume. This displayed a perfect use case of parallelization.

The command mpirun -np 4 apptainer exec ... reliably launched four independent container instances, each reporting a distinct MPI rank (0–3), even though the container image itself contained no MPI installation. This confirmed that parallel orchestration and the container's internal contents are fully decoupled in this model.

Two workstation-specific obstacles were identified and resolved. First, MPI's slot check rejected more processes than the number of physical cores it detected; this was resolved using --oversubscribe and an explicit slot specification. Second, MPI's attempt to pin each process to a distinct physical core failed for the same reason; this was resolved using --bind-to none. Both issues stem from the workstation's hyperthreaded CPU, which exposes more logical cores (as reported by nproc) than physical ones (as reported by lscpu). This distinction does not need to be managed manually on the cluster, where Slurm automatically allocates genuinely free physical cores.

On the cluster a 32-task Slurm array job (--array=0-31) calling the same function was submitted next. squeue confirmed that all 32 tasks were tracked (individually or as a condensed array range) and running concurrently. All resulting per-slice output images (ps_output_${SLURM_ARRAY_TASK_ID}.png) and job-logs (slurm-%A_%a.out) were saved automatically to the cluster. No errors were reported.

3.4 GPU-Accelerated Execution

A PyTorch test container was built successfully both against CUDA (gpu_test_nvidia.sif) and against ROCm (gpu_test_amd.sif).

Run locally without a dedicated GPU, torch.cuda.is_available() correctly and predictably returned False, confirming that the container itself was functionally correct and ready for the --nv/--rocm test on cluster hardware.

Submitted as a Slurm batch job requesting one GPU on the NVIDIA gpu partition, the container correctly reported GPU availability, the expected accelerator name, and the CUDA backend once executed with apptainer exec –nv.
The equivalent AMD job on the single-node agpu partition, executed with --rocm, correctly reported the ROCm/HIP backend via PyTorch's torch.version.hip attribute despite using the same torch.cuda.* interface for both vendors.

A subsequent matrix-multiplication workload executed in both jobs on the GPU, wrote its 5000×5000 result tensor to a bound output path, and was successfully reloaded and inspected afterwards.

For more detailed information on the report of connectivity with GPU-hardware and the test job see Figure 5 in the Appendix.

4. Discussion

4.1 Portability and Reproducibility

The central premise of using Apptainer held throughout this report: a single SIF file, once built and validated, can be moved to a Slurm-managed HPC cluster and executed there without administrative privileges or modification. This matches Apptainer's design goal of "mobility of compute" (Kurtzer et al., 2017).

The rootless, non-setuid installation, together with uidmap-based user namespaces, meant that only the initial container build required elevated (fakeroot) privileges. Every subsequent execution, on the workstation and on the cluster, ran under the invoking user's ordinary account (Naweiluo Zhou, 2022). In practice, this removed what would otherwise have been a recurring dependency on cluster administrators for every environment change (Kurtzer, 2017). The trade-off is additional discipline: the versioned .def file must be maintained as the single source for the environment, rather than relying on an interactively modified installation (Apptainer Project, n.d.).

4.2 Code/Data Separation as an HPC Design Pattern

Separating code from data through bind mounts, rather than embedding data in the image, proved to be more than a convenience (Apptainer Project, n.d.). It avoided rebuilding the container for every new dataset, kept the image small, and, importantly for a pipeline that may eventually process clinical or otherwise sensitive measurements, kept potentially sensitive data entirely outside of the versioned, shareable container artifact (Kurtzer, 2017).

This matches how bind mounts are commonly used in HPC settings, where project- and scratch-storage areas are deliberately not part of the default automatic binds and must instead be attached explicitly for each job (Apptainer Project, n.d.-b).

One practical trade-off surfaced during this work, concerning how the container learns where to find externally bound data. The first approach hardcodes a fixed internal path in the pipeline (e.g., /data/input) and binds external data to that same path at runtime (Apptainer Project, n.d.). This requires no code changes, since the pipeline always expects data at the same location, but it ties every future dataset to that fixed location and means anyone using the container must already know which internal path to bind to. The second approach, which 'spacemed' already uses for several entry points (Gruhlke, 2026), exposes the data path as an explicit script argument instead. The bind target is then chosen freely at runtime and passed to the script directly (e.g., --input /mnt/data), rather than being assumed by the code (Apptainer Project, n.d.). This requires slightly more explicit bookkeeping in the Slurm script, since the bind path and the script argument must be kept consistent, but it decouples the pipeline entirely from any single container-internal convention and makes the data location visible at the point of invocation rather than hidden inside the code.

As this work was purely educational, the bind-mount couldn’t be fully tested in a way where an external container was used to bind to the user’s home directory or the other way around. Meaning, while the jobs ran and the concept of bind mount was proven, a bind mount would not have been technically necessary.

4.3 Parallelization Trade-offs

Keeping the container single-process, with no awareness of parallelism, and instead letting the host's mpirun or Slurm's array-job mechanism start multiple instances, avoids a specific problem: having to match the MPI version inside the container to the specialized MPI stack the cluster provides (Apptainer Project, n.d.). For workloads like 'spacemed', where independent datasets or independent fMRI slices can simply be processed side by side, this host-level approach worked well and needed no changes to the container between running it as a single process or as many parallel ones (Gruhlke, 2026). Installing MPI inside the container remains a sensible option only when the container needs to stay fully self-contained and run on systems where no compatible MPI is available on the host (Krishna Kant Singh, 2026).

4.4 GPU Virtualization and the Reproducibility Compromise

Apptainer's --nv/--rocm mechanism is best understood as a second, specialized bind mount, one that exposes host driver libraries and device files instead of user data, rather than as a departure from the bind-mount model used for data (Apptainer Project, n.d.-d). This is a deliberate, acknowledged compromise. The whole point of a container is that its operating system and library versions stay frozen (Kurtzer, 2017); but GPU acceleration reintroduces one host-specific component into that otherwise frozen image: the driver. This works in practice because NVIDIA and AMD maintain driver/userspace compatibility with the CUDA or ROCm toolkit built into the container (NVIDIA Corporation, n.d.; AMD, n.d.).

Producing a correct result once, on one GPU, is not enough to call a container "GPU-portable." Instead, it needs to work consistently across the different accelerator types available on the cluster, checking both that the GPU is detected (torch.cuda.is_available()) and that a real computation runs correctly on each one (A100, L40S, and H100/H200 using --nv, and

MI210 using --rocm). This poses a constraint on the reproducibility and compatibility: Different containers need to be built for different GPUs, even though potentially running the same program (Apptainer Project, n.d.).

4.5 Limitations

This work is a single, descriptive case report of one pipeline and one cluster; the workflow's applicability to substantially different software stacks (for example compiled C/C++ or R-based pipelines rather than a Conda-based Python package) was not tested. Verification throughout emphasized correctness (did the container build, did the expected files appear, did the expected rank or device name print) was performed, rather than systematic performance benchmarking: No scaling curves (runtime versus number of parallel tasks) were collected, so the point at which additional parallelism stops paying off for ‘spacemed''s actual workloads remains an open empirical question rather than a measured one. The GPU validation likewise used a generic matrix-multiplication placeholder rather than a real ‘spacemed’ analysis, since no GPU-dependent analysis step exists in the pipeline yet. Finally, although the motivation for separating code from data explicitly anticipated eventually handling sensitive or clinical measurements, this report did not involve a formal review of Charité's data-protection requirements for such data (storage location, network isolation during jobs, permitted bind mounts); any deployment intended for real patient-adjacent data would need to be assessed with Charité's IT and data-protection functions before use, as cluster-specific policies of this kind cannot be inferred from the container tooling alone.

5. Outlook and Perspectives

The product of this work is an instruction and introduction to working with Apptainer on the Charité-HPC-cluster, but it is not exhaustive and many more use-cases and systems are still to cover. First of all, all instructions are based on the examples of the ‘spacemed’ project (Python-based) on the Charité’s HPC cluster. Other projects with different programming languages or different cluster architecture are not covered. This becomes apparent in terms of CPU-architecture: The cluster runs on x86_64 and containers have to be built on the same CPU-architecture at the current state of work; e.g. ARM-based container builds are not covered (Kurtzer, 2017). Other extensions also follow naturally from the current state of work.

First, the correctness-focused validation performed here should be complemented with systematic scaling benchmarks, to establish, rather than assume, the point of diminishing returns for ‘spacemed's’ actual workloads on the cluster's compute nodes. This could be complemented by a script reporting the estimated workload to the user and therefore helping to choose suitable hardware even before running a container to help with more efficient resource allocation.

Second, before any clinically derived data is processed on the cluster through this container, the data-handling design should be reviewed together with Charité's HPC operations and data-protection functions, covering storage location, permitted bind-mount targets, and network policy during job execution.

Third, it would be worthwhile to revisit the in-container MPI approach once a workload arises that must remain portable to systems without a compatible host MPI stack, since the host-level model favoured in this report is well suited for the Charité cluster but is not the only supported approach (Apptainer Project, n.d.; Krishna Kant Singh, 2026).

Fourth, the container definition itself can be extended with the optional %test and %help sections to make the build self-verifying and self-documenting (Apptainer Project, n.d.), and the Sphinx dependency already present in ‘spacemed.yml’ suggests that generating proper

package documentation is a natural next housekeeping step (Gruhlke, 2026). Along the same lines, pipeline automation for container-build, e.g. using nextflow could simplify the build-process.
Therefore the current project is a good instruction to starting to work with Apptainer and using it in everyday work, but more advanced use cases are not covered yet.

References

(30.08.2026).
AMD. (n.d.). *User and AMD GPU driver (amdgpu) support matrix. ROCm Documentation.* Retrieved 08 2026, from https://rocm.docs.amd.com/projects/install-on-linux/en/latest/reference/user-kernel-space-compat-matrix.html
Apptainer Project. (n.d.). *Apptainer User Guide. Apptainer and MPI applications.* Retrieved 08 2026, from https://apptainer.org/user-docs/master/mpi.html
Apptainer Project. (n.d.). *Apptainer User Guide. Definition files.* Retrieved 08 2026, from https://apptainer.org/docs/user/main/definition_files.html
Apptainer Project. (n.d.). *Apptainer User Guide. GPU and other device support.* Retrieved 08 2026, from https://apptainer.org/docs/user/main/gpu.html
Apptainer Project. (n.d.). *Installing Apptainer. Apptainer Admin Guide.* Retrieved 08 2026, from https://apptainer.org/docs/admin/main/installation.html
Apptainer Project. (n.d.). *User Guide. Bind paths and mounts.* Retrieved 08 2026, from https://apptainer.org/docs/user/main/bind_paths_and_mounts.html
Charité Universitätsmedizin Berlin. (n.d.). *Charité Scientific Computing (SC) HPC Wiki. [Internal documentation, restricted to the Chari-té network].* Retrieved 08 2026, from https://git.bihealth.org/charite-sc-public/sc-wiki/-/wikis/home
Charité Universitätsmedizin Berlin. (n.d.). *Science IT: Scientific Computing | HPC.* Retrieved 08 2026, from https://www.charite.de/en/research/research_support_services/research_infrastructure/science_it
David Moreau, K. W. (2023, 07 13). Containers for computational reproducibility. *Nature Reviews Methods Primers*. doi:10.1038/s43586-023-00236-9
Gruhlke, K. (2026, 08). *GitHub*. Retrieved from ‘spacemed’: A Python package for pulse-oximetry peak detection and heart-rate estimation [Software repository].: https://github.com/KiraGruhlke/’spacemed’
Krishna Kant Singh, E. M. (2026, 03 12). HPC Containers for EBRAINS: Towards Portable Cross-Domain Software Environment. doi:10.48550
Kurtzer, G. M. (2017). Singularity: Scientific containers for mobili-ty of compute. *PLOS ONE, 12(5), e0177459.*, https://doi.org/10.1371/journal.pone.0177459.
Naweiluo Zhou, H. Z. (2022, 12 16). Containerisation for High Performance Computing Systems: Survey and Prospects. doi:10.48550
NVIDIA Corporation. (n.d.). *CUDA compatibility. NVIDIA Documentation Hub.* Retrieved 08 2026, from https://docs.nvidia.com/deploy/cuda-compatibility/latest/

Academic integrity declaration

I hereby declare that I have written all parts of this thesis independently and that I have not used any sources other than those indicated in the thesis and that I have not submitted the thesis in the same or a similar form for any other examination. All verbatim or analogous copies and quotations have been identified and verified.

Information on the use of AI-based tools

I affirm that I have not used any AI-based tools whose use has been explicitly excluded in writing by the examiner. I am aware that the use of texts or other content and products generated by AI-based tools does not guarantee their quality. I am fully responsible for the adoption of any machine-generated passages used by me and bear the responsibility for any erroneous or distorted content, faulty references, violations of data protection and copyright law or plagiarism generated by the AI. I also affirm that my creative influence predominates in the present work.

Hamburg, 30.08.2026 

(Place, Date, Signature)

Appendix

Table 1. GPU partitions on the Charité SC cluster and the corresponding Apptainer flag (Charité Universitätsmedizin Berlin, n.d.).

Node(s)	Partition	Accelerator (GRES)	Per node	Flag
s-sc-gpu[002-021]	gpu	NVIDIA A100-PCIe 40 GB	1	--nv
s-sc-gpu[022-028]	gpu	NVIDIA A100 80 GB PCIe	1	--nv
s-sc-gpu029	gpu	NVIDIA L40S	2	--nv
s-sc-pgpu08	pgpu	NVIDIA H100 80 GB HBM3	8	--nv
s-sc-dgx[01-02]	pgpu	NVIDIA A100-SXM4 80 GB	8	--nv
s-sc-pgpu[03-07]	pgpu	NVIDIA A100-SXM4 80 GB	4	--nv
s-sc-pgpu[11-15]	pgpu	NVIDIA H200	8	--nv
s-sc-pgpu16	pgpu	NVIDIA H200	4	--nv
s-sc-pgpu[01-02]	pgpu	NVIDIA A100-SXM4 40 GB	4	--nv
s-sc-agpu001	agpu	AMD Instinct MI210 (gfx90a)	1	--rocm

Table 2. Selected Build- and Runtime Diagnostics. The following diagnostics were encountered during container builds and job execution and are recorded here as supplementary reference material; none required a change to the overall workflow described in Section 2.

Message / situation	Interpretation
pip warning: "Running pip as the 'root' user..."	Generic pip caution issued during the %post build step; inconsequential because the installation targets an isolated Conda environment inside a disposable, fakeroot build context, not the host system.
pip retry / ReadTimeoutError while downloading an NVIDIA CUDA wheel	Transient network timeout during a large download; resolved automatically by pip's built-in retry logic on the next attempt.
Apptainer build fails: session directory too small	Apptainer's default in-memory session size (64 MB) was exceeded; resolved by increasing 'sessiondir max size' in the Apptainer configuration or by redirecting build temp storage with APPTAINER_TMPDIR to a directory on a larger partition.
"Authorization required, but no authorization protocol specified"	Matplotlib attempted to open an interactive display window inside a headless container; resolved by writing to a file explicitly (-o) and/or forcing the non-interactive Agg backend via MPLBACKEND=Agg.
ImportError: attempted relative import	A package-internal script was invoked directly as a file instead of as a module; resolved by calling it with python -m <package>.<module>.
MPI error: not enough slots available	mpirun requested more processes (-np) than the detected number of usable slots (physical cores); resolved for local testing with --oversubscribe or by specifying slots explicitly (--host localhost:N).
MPI error / hang related to CPU binding	mpirun could not bind each process to a distinct physical core once physical cores were exhausted; resolved for local testing with --bind-to none.
torch.load fails: "...CUDA-GPU..., but torch.cuda.is_available() is False"	A tensor that was saved from a GPU session was reloaded in a session without GPU access; resolved by re-running with GPU access, or by loading with map_location=torch.device('cpu') for inspection only.

```
# Testet, ob PyTorch GPU-Zugriff hat (NVIDIA via CUDA oder AMD via ROCm)
# und führt eine echte Rechenoperation auf der GPU durch.

import argparse
import torch

# Pruefen, ob ueberhaupt eine GPU verfuegbar ist
gpu_available = torch.cuda.is_available()
print("GPU available:", gpu_available)
# Namen der zugewiesenen GPU anzeigen
if gpu_available:
    print("Device name:", torch.cuda.get_device_name(0))
    # Zeigt an, ob PyTorch intern mit CUDA (NVIDIA) oder ROCm/HIP (AMD) läuft
    if torch.version.hip is not None:
        print("Backend: ROCm (AMD)")
    else:
        print("Backend: CUDA (NVIDIA)")
else:
    print("Keine GPU erkannt - entweder ist dieser Container nicht mit")
    print("GPU-Unterstützung gebaut, oder es lief kein --nv/--rocm Flag mit.")

# Rechengerät festlegen: GPU falls verfügbar, sonst CPU als Rückfalloption. Genutztes Geraet ausgeben.
device = torch.device("cuda" if gpu_available else "cpu")
print("Using device:", device)

# Zwei zufällige 5000x5000-Matrizen direkt auf dem gewählten Gerät erzeugen
a = torch.rand(5000, 5000, device=device)
b = torch.rand(5000, 5000, device=device)

# Matrixmultiplikation ausführen - das ist die eigentliche Rechenlast,
# die bei einer GPU deutlich schneller laufen sollte als auf der CPU
result = torch.matmul(a, b)

# Kurze Kontrollausgaben, um das Ergebnis grob zu überprüfen
print("Result shape:", result.shape) # sollte wieder eine 5000*5000 Matrix sein.
print("Result sum:", result.sum().item()) # addiert alle Einzelwerte und formatiert sie zu einer normalen zahl, sollte sehr hoch sein.

# Ergebnis als Datei abspeichern (.pt ist das Standardformat
# für gespeicherte PyTorch-Tensoren/Modelle)
# Ausgabepfad wird über das -o Flag übergeben, statt fest im Code zu stehen -
# so funktioniert das Skript für jeden Nutzer, unabhängig vom Home-Verzeichnis.
parser = argparse.ArgumentParser(description="GPU-Matrixmultiplikationstest mit PyTorch")
parser.add_argument("-o", "--output", default="matmul_result.pt",
                    help="Pfad zur Ausgabedatei (Standard: matmul_result.pt)")
args = parser.parse_args()

torch.save(result, args.output)
print("Saved result to", args.output)
```

Figure 5. Python script used to verify GPU accessibility and functional correctness inside the container. The script checks whether PyTorch detects a GPU via torch.cuda.is_available(), reports the backend in use (CUDA for NVIDIA or ROCm/HIP for AMD), and performs a 5000×5000 matrix multiplication on the detected device to confirm that computation runs correctly, before saving the result to a user-specified output path.

CONTAINERIZING A SCIENTIFIC
DATA-ANALYSIS PIPELINE
FOR HPC DEPLOYMENT

www.charite.de